

Modul 324 DevOps

React Projekt mit Package

Johnny Leonhardt, 5IA23b, 11.09.2026

Repositories:

- React-Package (Quellcode): github.com/Johnsoryna/m324-react-bargraph
- Publiziertes Package: npmjs.com/package/@johnsoryna/mybargraph
- Konsument (eigenes Package + Package des Lernpartners): github.com/Johnsoryna/m324-react-consumer

Umgebung: Windows 11, Node 22.16.0, npm 11.4.2, Vite 8.3, React 19.2, Git 2.48, IntelliJ IDEA 2024.3

Aufgabe: Recherche: React-Komponente als Package bereitstellen

Ein React-Package ist ein normales npm-Package. Der Unterschied zu einer React-App: Es hat keine `index.html` und kein `main.jsx`, sondern einen Einstiegspunkt (`src/index.js`), der die Komponenten exportiert. Ein Bundler baut daraus fertige JavaScript-Dateien in `dist/`. Der Konsument bekommt nicht den Quellcode, sondern dieses Artefakt aus `node_modules`, genau wie beim Maven-Artefakt in der letzten Aufgabe das JAR.

Diese Punkte muss ein React-Package erfüllen:

Punkt	Bedeutung
Einstiegspunkt <code>src/index.js</code>	Exportiert, was das Package nach aussen anbietet (Default- und/oder Named-Exports).
Build-Output <code>dist/</code>	JSX ist bereits nach JavaScript übersetzt. ES-Modul für Vite/webpack, CommonJS für <code>require()</code> und Jest.
<code>main / module / exports</code>	Sagen npm und dem Bundler, welche Datei aus <code>dist/</code> beim Import geladen wird.
<code>files</code>	Nur <code>dist/</code> (plus README, <code>package.json</code>) landet im Package, nicht Quellcode, Demo oder <code>node_modules</code> .
<code>peerDependencies</code>	react und react-dom werden nicht mitgeliefert, das Zielprojekt hat sie schon. Sonst gibt es zwei React-Instanzen und Hooks funktionieren nicht.
CSS	Muss ins Bundle injiziert werden (<code>vite-plugin-lib-inject-css</code>). Sonst muss der Konsument das CSS separat importieren.
Version	Semantic Versioning. Jede Publikation braucht eine neue, höhere Versionsnummer.

Zum Bauen gibt es mehrere Werkzeuge. Das Medium-Tutorial aus dem Auftrag ist hinter einer Paywall, sichtbar ist nur die Einleitung. Der nPlan-Artikel (Rollup + GitHub Packages) ist vollständig lesbar und zeigt den Rollup-Weg.

Werkzeug	Vorgehen	Vorteile	Nachteile
Vite Library-Modus	<code>build.lib</code> in <code>vite.config.js</code> ,	Gleiche Toolchain wie die	Ein Package pro Projekt,

	Formate es + cjs, react als external	App, wenig Konfiguration, Dev-Server für eine Demo inklusive	mehrere Packages aus einem Repo sind mühsam
Rollup direkt (nPlan-Artikel)	rollup.config.js mit Babel, commonjs, node-resolve, postcss, peer-deps-external	Volle Kontrolle, mehrere Packages aus einem Repo möglich	Viele Plugins, viel Konfiguration, Babel separat einrichten
tsup / esbuild	Ein Befehl: tsup src/index.js -format esm,cjs	Sehr schnell, fast keine Konfiguration	CSS-Handling schwach, eher für TypeScript-Libraries ohne Styles
Ohne Build (JSX publizieren)	Quellcode direkt in files aufnehmen	Nichts zu bauen	Konsument muss JSX aus node_modules selbst kompilieren, das tun Vite/webpack standardmässig nicht. Unüblich.

Entscheidung: Vite Library-Modus. Das Projekt ist mit Vite erstellt, die Anleitung im Modul baut darauf auf, und der Dev-Server lässt sich für eine Demo der Komponente weiterverwenden.

Aufgabe: Recherche: Alternative: publish on GitHub

GitHub Packages enthält neben der Maven-Registry auch eine npm-Registry unter `npm.pkg.github.com`. Das Vorgehen ist das gleiche wie bei GitHub Maven Packages in der letzten Aufgabe: Token mit `write:packages`, Registry-URL angeben, publizieren. Die Unterschiede zu `npmjs.com` liegen im Detail:

- Der Package-Name muss mit dem GitHub-Benutzer als Scope beginnen: `@johnsoryna/mybargraph`.
- Eine `.npmrc` im Projekt verbindet den Scope mit der Registry und liefert das Token. Alternativ `publishConfig.registry` in der `package.json`.
- Das Feld `repository.url` in der `package.json` muss auf das GitHub-Repository zeigen, sonst lehnt die Registry das Package ab.
- Der Token ist ein Personal Access Token (classic) mit `write:packages` (enthält `read:packages`) und `repo`.
- Auch der Konsument braucht eine `.npmrc` mit Scope-Mapping und einem Token mit `read:packages`, sogar bei öffentlichen Packages. Das ist der grösste Unterschied zu `npmjs.com`.

Konfiguration für GitHub Packages (für das eigene Package nicht umgesetzt; der Lese-Teil kam beim Package des Lernpartners zum Einsatz, siehe unten):

```
# .npmrc (Package-Projekt und Konsument)
@johnsoryna:registry=https://npm.pkg.github.com
//npm.pkg.github.com/:_authToken=${GITHUB_TOKEN}

# package.json (Package-Projekt)
"publishConfig": { "registry": "https://npm.pkg.github.com" },
"repository": { "type": "git", "url": "git+https://github.com/Johnsoryna/m324-react-bargraph.git" }

npm publish
```

Kriterium	npmjs.com	GitHub Packages
Sichtbarkeit	Öffentlich für alle, private Packages kostenpflichtig	Public oder private, an Repository gebunden
Installation beim Konsument	npm install, keine Konfiguration	.npmrc mit Scope und Token nötig, auch bei public
Authentifizierung beim Publizieren	npm login (Browser) oder Access-Token, bei 2FA zusätzlich OTP	Personal Access Token mit write:packages
Kosten	Gratis für public	Gratis für public, Speicher- und Traffic-Limit bei private
Versionen löschen	Version unveränderlich, unpublish nur innerhalb 72 h	Versionen können gelöscht werden
CI/CD	Trusted Publishing (OIDC) oder Token als Secret	GITHUB_TOKEN in Actions direkt nutzbar
Eignung für Lernpartner	Einfach, jeder kann sofort installieren	Jeder Partner braucht einen GitHub-Token

Entscheidung: npmjs.com. Die Lernpartner können das Package ohne Token installieren. GitHub Packages passt, wenn ein Package privat bleiben soll und alle Konsumenten sowieso in derselben GitHub-Organisation sind, wie im nPlan-Artikel beschrieben.

Aufgabe: React-Package erstellen

Ziel: Die Komponente HorizontalBarGraph als npm-Package @johnsoryna/mybargraph, gebaut mit Vite im Library-Modus.

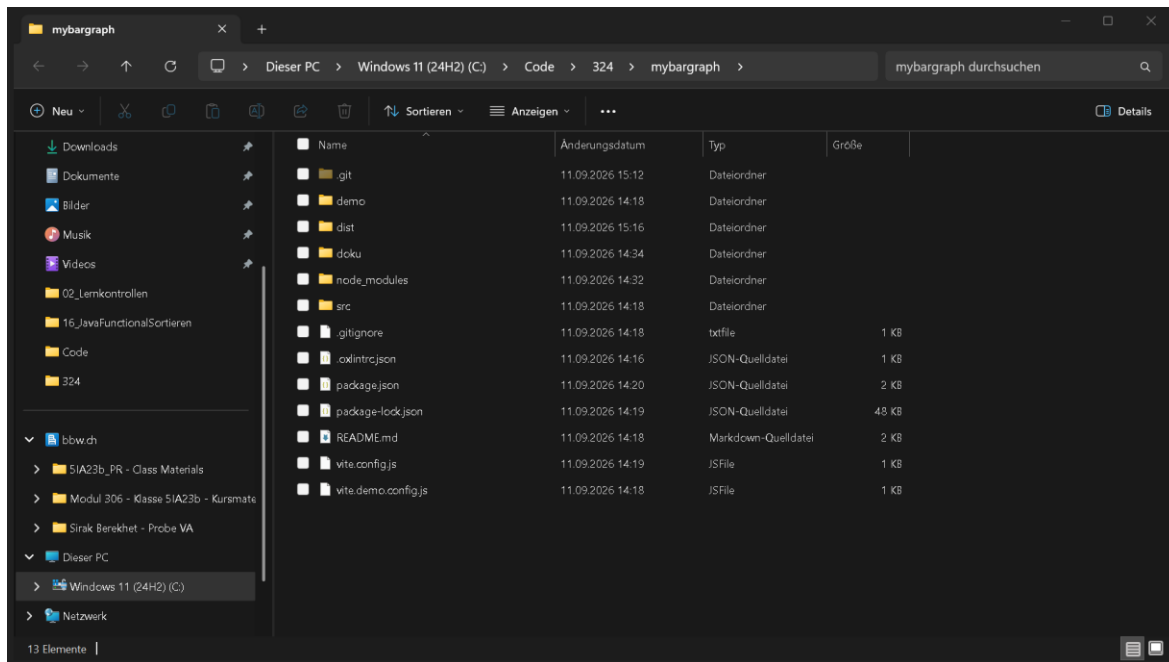
Schritt 1: Projekt anlegen und Template ausmisten

Vite kann kein Library-Projekt direkt erzeugen. Darum zuerst ein normales React-Projekt (IntelliJ: File > New > Project > Vite, Framework React, JavaScript; im Terminal gleichwertig):

```
npm create vite@latest mybargraph -- --template react
cd mybargraph
npm install
```

Danach alles entfernt, was nur eine App braucht: index.html, public/, src/App.jsx, src/App.css, src/index.css, src/main.jsx, src/assets. Übrig bleibt diese Struktur:

```
mybargraph/
  package.json          Name, Version, files, exports, peerDependencies
  vite.config.js         Library-Modus (Build nach dist/)
  vite.demo.config.js    nur für die lokale Demo
  src/index.js           Einstiegspunkt, exportiert die Komponente
  src/components/HorizontalBarGraph.jsx
  src/components/HorizontalBarGraph.css
  demo/index.html, demo/src/main.jsx Demo-App (nicht im Package)
  dist/                 Build-Ergebnis (in .gitignore, aber im Package)
```



Projektstruktur des Packages: src, demo, dist, doku und die beiden Vite-Konfigurationen

Schritt 2: vite.config.js auf Library-Modus umstellen

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import { fileURLToPath } from 'node:url'
import { libInjectCss } from 'vite-plugin-lib-inject-css'

export default defineConfig({
  plugins: [react(), libInjectCss()],
  build: {
    lib: {
      entry: fileURLToPath(new URL('./src/index.js', import.meta.url)),
      name: 'MyBargraph',
      formats: ['es', 'cjs'],
      fileName: (format) => `mybargraph.${format}.js`,
    },
    rollupOptions: {
      external: ['react', 'react-dom', 'react/jsx-runtime', 'prop-types'],
      output: { exports: 'named', globals: { react: 'React', 'react-dom': 'ReactDOM' } },
    },
  },
})
```

Zwei Abweichungen von der Anleitung im Modul: Die package.json hat "type": "module", darum gibt es `__dirname` nicht, ich verwende `fileURLToPath(new URL(...))`. Und `exports: 'named'` verhindert die Warnung `MIXED_EXPORTS`, weil `src/index.js` sowohl einen Default- als auch einen Named-Export hat. `libInjectCss` schreibt einen import `./index.css` in das gebaute Modul, so lädt der Konsument das CSS automatisch mit.

Schritt 3: package.json anpassen

```
{
  "name": "@johnsoryna/mybargraph",
  "version": "1.0.0",
```

```

"private": false,
"type": "module",
"license": "MIT",
"repository": { "type": "git", "url": "git+https://github.com/Johnsoryna/m324-react-bargraph.git" },
"files": ["dist"],
"main": "./dist/mybargraph.cjs.js",
"module": "./dist/mybargraph.es.js",
"exports": {
  ".": { "import": "./dist/mybargraph.es.js", "require": "./dist/mybargraph.cjs.js" }
},
"sideEffects": ["**/*.css"],
"scripts": { "build": "vite build", "demo": "vite --config vite.demo.config.js" },
"peerDependencies": { "react": ">=18", "react-dom": ">=18" },
"dependencies": { "prop-types": "^15.8.1" },
"devDependencies": { "react": "^19.2.8", "react-dom": "^19.2.8", "vite": "^8.3.0",
  "@vitejs/plugin-react": "^6.1.1", "vite-plugin-lib-inject-css":
  "^2.2.2" }
}

```

- Der Name trägt den npm-Benutzernamen als Scope (@johnsoryna/), damit er eindeutig ist.
- react und react-dom stehen in peerDependencies (der Konsument liefert sie) und zusätzlich in devDependencies, damit Build und Demo lokal funktionieren.
- files: ["dist"] sorgt dafür, dass Quellcode und Demo nicht im Package landen.
- sideEffects: ["**/*.css"] verhindert, dass ein Bundler den CSS-Import beim Tree-Shaking entfernt.

Schritt 4 und 5: Einstiegspunkt und Komponente

```

// src/index.js
export { default } from './components/HorizontalBarGraph.jsx'
export { default as HorizontalBarGraph } from './components/HorizontalBarGraph.jsx'

// src/components/HorizontalBarGraph.jsx (gekürzt)
import PropTypes from 'prop-types'
import './HorizontalBarGraph.css'

export default function HorizontalBarGraph({ value, maxvalue = 100, barwidthpx = 300,
  label = '', color = '#2563eb', showValue = true
}) {
  const safeMax = maxvalue > 0 ? maxvalue : 1
  const clamped = Math.min(Math.max(value, 0), safeMax)
  const percent = Math.round((clamped / safeMax) * 100)
  return (
    <div className="hbg" style={{ width: `${barwidthpx}px` }}>
      <div className="hbg__header"><span>{label}</span>
        {showValue && <span>{clamped} / {safeMax} ({percent}%)</span></div>
      <div className="hbg__track" role="progressbar" aria-valuenow={clamped} aria-
        valuemax={safeMax}>
        <div className="hbg__bar" style={{ width: `${percent}%`, backgroundColor: color }}>
        </div>
      </div>
    </div>
  )
}
HorizontalBarGraph.propTypes = { value: PropTypes.number.isRequired, maxvalue:
  PropTypes.number, ... }

```

Der Balken wächst per CSS-Transition auf die neue Breite, wenn sich value ändert. Props der Komponente:

Prop	Typ	Default	Beschreibung
value	number	(Pflicht)	Aktueller Wert, wird auf 0..maxvalue begrenzt
maxvalue	number	100	Maximalwert, entspricht 100 %
barwidthpx	number	300	Gesamtbreite des Balkens in Pixel
label	string	"	Beschriftung über dem Balken
color	string	'#2563eb'	Farbe des gefüllten Balkens
showValue	boolean	true	Wert und Prozentzahl rechts anzeigen

Schritt 6 und 7: prop-types installieren und bauen

```
npm install prop-types --save
npm install -D vite-plugin-lib-inject-css
npm run build
```

```
dist/index.css      0.51 kB
dist/mybargraph.es.js 1.32 kB
dist/mybargraph.cjs.js 1.57 kB
built in 126ms
```

Die Bundles sind klein, weil react, react-dom und prop-types nicht enthalten sind. npm pack --dry-run zeigt, was tatsächlich publiziert würde:

```
npm notice package: @johnsoryna/mybargraph@1.0.0
npm notice 1.7kB README.md
npm notice 514B dist/index.css
npm notice 1.6kB dist/mybargraph.cjs.js
npm notice 1.3kB dist/mybargraph.es.js
npm notice 1.3kB package.json
npm notice package size: 3.0 kB, total files: 5
```

```
Windows PowerShell
computing gzip size...
dist/index.css      0.51 kB | gzip: 0.33 kB
dist/mybargraph.es.js 1.32 kB | gzip: 0.63 kB

✓ 4 modules transformed.
computing gzip size...
dist/index.css      0.51 kB | gzip: 0.33 kB
dist/mybargraph.cjs.js 1.57 kB | gzip: 0.80 kB

✓ built in 783ms
npm notice
npm notice package: @johnsoryna/mybargraph@1.0.0
npm notice Tarball Contents
npm notice 1.7kB README.md
npm notice 511B dist/index.css
npm notice 1.6kB dist/mybargraph.cjs.js
npm notice 1.3kB dist/mybargraph.es.js
npm notice 1.3kB package.json
npm notice Tarball Details
npm notice name: @johnsoryna/mybargraph
npm notice version: 1.0.0
npm notice filename: johnsoryna-mybargraph-1.0.0.tgz
npm notice package size: 3.0 kB
npm notice unpacked size: 6.5 kB
npm notice shasum: 11582505aa0113dbd2c365989a1aad4421fc9987
npm notice integrity: sha512-FQsLhRMJTrsY+[...]mxFt7wiE2AKrQ==
npm notice total files: 5
npm notice
johnsoryna-mybargraph-1.0.0.tgz
PS C:\Code\324\mybargraph>
```

npm run build und npm pack --dry-run im Terminal

Demo im Package selbst

Für die Demo gibt es den Ordner demo/ mit index.html und src/main.jsx, die die Komponente direkt aus src/ importiert. Das Vorgehen aus der Anleitung ("demo": "vite demo") hat bei mir nicht funktioniert: Vite sucht die vite.config.js im angegebenen Root, also in demo/, findet sie nicht und hat dann kein React-Plugin. Darum eine eigene kleine Konfiguration:

```
// vite.demo.config.js
export default defineConfig({ root: 'demo', plugins: [react()] })

// package.json
"demo": "vite --config vite.demo.config.js"
```

MyBargraph – Demo



npm run demo: Demo-Seite mit Slider und drei Balken im Browser

Aufgabe: Package publizieren (npmjs.com)

Voraussetzung ist ein Account auf npmjs.com, der Benutzername (johnsoryna) ist der Scope im Package-Namen. Für die Authentifizierung gibt es zwei Wege:

Weg	Vorgehen	Bewertung
npm login	Öffnet den Browser, man meldet sich an, das Token landet in ~/.npmrc des Benutzers	Kein Token im Projekt, für die manuelle Abgabe ideal
Access-Token (Anleitung im Modul)	Token auf npmjs.com erzeugen, in .npmrc im Projekt eintragen, .npmrc in .gitignore	Nötig für CI/CD; Risiko, dass das Token versehentlich committet wird

Ich habe npm login gewählt. Die Datei .npmrc steht trotzdem in .gitignore, falls später ein Token für eine Pipeline dazukommt. Publiziert wird mit --access public, weil Packages mit Scope standardmässig privat wären und private Packages auf npmjs.com kostenpflichtig sind.

Der erste Versuch scheiterte mit 403: npmjs.com verlangt seit 2025 für jedes Publizieren eine Zwei-Faktor-Authentifizierung, Tokens mit "Bypass 2FA" (wie in der Modul-Anleitung beschrieben) werden eingeschränkt. Nach dem Aktivieren von 2FA mit einem Passkey öffnet npm publish einen Browser-Link,

dort bestätigt man mit dem Passkey, und das Terminal fährt fort. Wichtig: Das funktioniert nur in einem interaktiven Terminal, nicht in einem Skript.

```
PS C:\Code\324\mybargraph> npm login
Logged in on https://registry.npmjs.org/.

PS C:\Code\324\mybargraph> npm publish --access public
npm notice package: @johnsoryna/mybargraph@1.0.0
npm notice total files: 5
npm notice Publishing to https://registry.npmjs.org/ with tag latest and public access
Authenticate your account at:
https://www.npmjs.com/auth/cli/984ba77b-....
Press ENTER to open in the browser...

+ @johnsoryna/mybargraph@1.0.0
```



npm tokens that bypass 2FA are being restricted — account changes (Aug 2026) and direct publishing (Jan 2027). [Learn how to prepare](#) →

Pro Teams Pricing Documentation

Search packages

@johnsoryna/mybargraph
1.0.0 • Public • Published 7 minutes ago

Readme Code (Beta) 1 Dependency 0 Dependents 1 Versions Settings

@johnsoryna/mybargraph

Eine einfache, animierte React-Komponente `HorizontalBarGraph`, die einen Wert als horizontalen Balken relativ zu einem Maximalwert anzeigt. Entstanden im Modul 324 DevOps (Thema: Module / Komponenten / Artefakte).

Installation

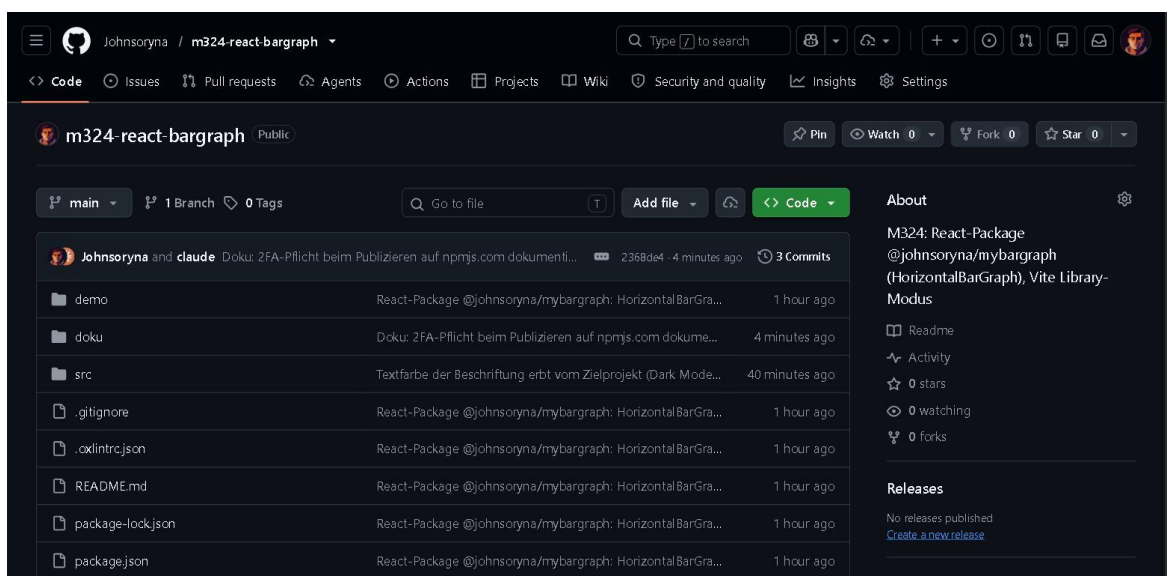
Install

```
> npm i @johnsoryna/mybargraph
```

Repository

github.com/Johnsoryna/m324-react-bargraph

Das Package @johnsoryna/mybargraph auf npmjs.com, Version 1.0.0, public, mit Link zum Repository



Johnsoryna / m324-react-bargraph

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

m324-react-bargraph Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file Code

Johnsoryna and claude Doku: 2FA-Pflicht beim Publizieren auf npmjs.com dokumenti... 2368de4 · 4 minutes ago 3 Commits

File	Description	Time
demo	React-Package @johnsoryna/mybargraph: HorizontalBarGra...	1 hour ago
doku	Doku: 2FA-Pflicht beim Publizieren auf npmjs.com dokume...	4 minutes ago
src	Textfarbe der Beschriftung erbt vom Zielprojekt (Dark Mode...	40 minutes ago
.gitignore	React-Package @johnsoryna/mybargraph: HorizontalBarGra...	1 hour ago
.oxlint.json	React-Package @johnsoryna/mybargraph: HorizontalBarGra...	1 hour ago
README.md	React-Package @johnsoryna/mybargraph: HorizontalBarGra...	1 hour ago
package-lock.json	React-Package @johnsoryna/mybargraph: HorizontalBarGra...	1 hour ago
package.json	React-Package @johnsoryna/mybargraph: HorizontalBarGra...	1 hour ago

About

M324: React-Package
@johnsoryna/mybargraph
(HorizontalBarGraph), Vite Library-Modus

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published
[Create a new release](#)

Quellcode-Repository m324-react-bargraph auf GitHub

Eine publizierte Version ist unveränderlich. Für jede weitere Publikation muss die Version erhöht werden, z.B. mit `npm version patch (1.0.0 -> 1.0.1)`, erst dann geht `npm publish` wieder durch.

Aufgabe: Package im eigenen Projekt verwenden

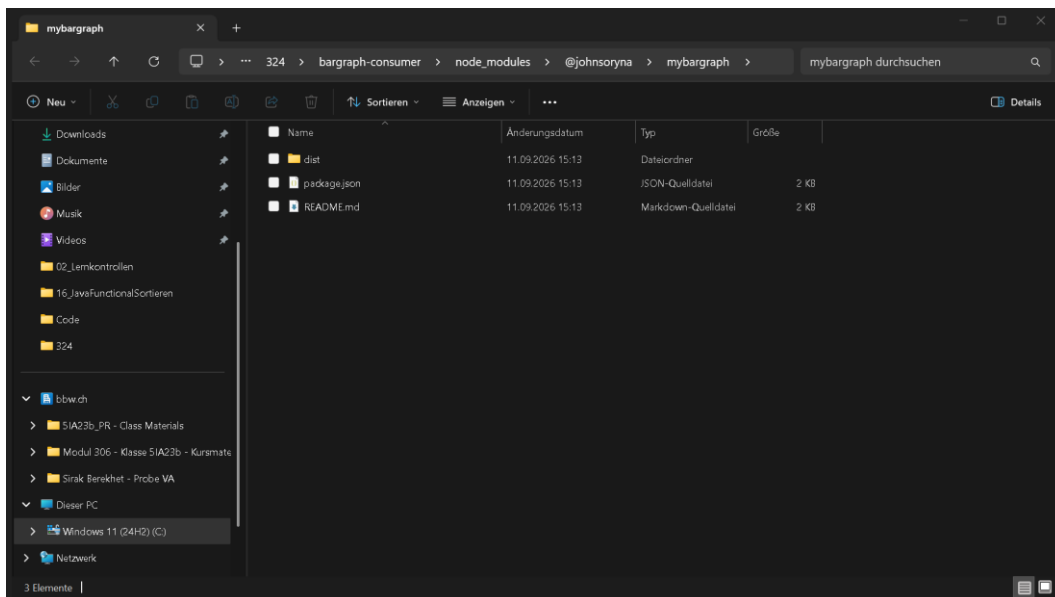
Zweites Projekt `bargraph-consumer`, ebenfalls mit Vite und React. Das Package wird wie jede andere Abhängigkeit installiert und landet unter `node_modules/@johnsoryna/mybargraph`, nur mit `dist/`, `README` und `package.json`, ohne Quellcode.

```
npm create vite@latest bargraph-consumer -- --template react
cd bargraph-consumer
npm install
npm install @johnsoryna/mybargraph

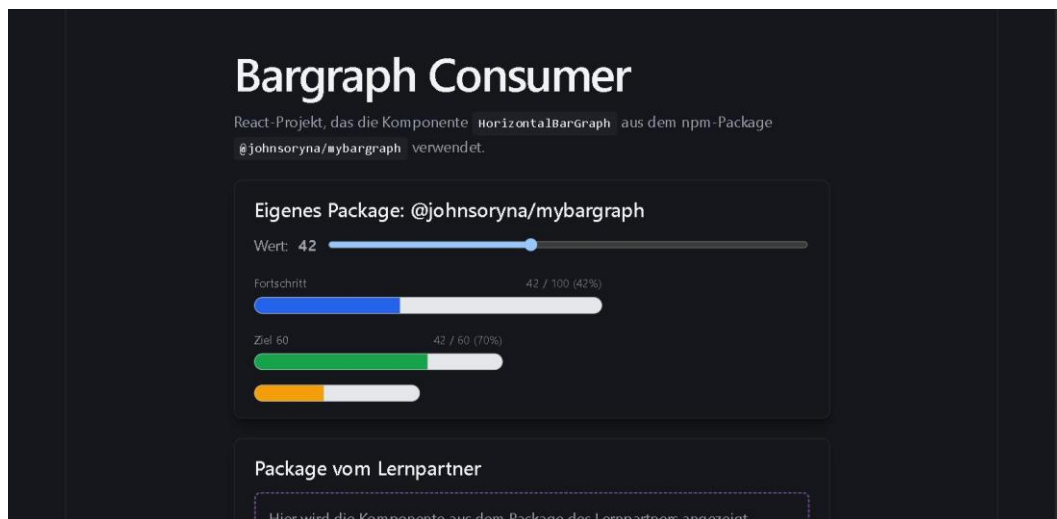
// src/App.jsx (gekürzt)
import { useState } from 'react'
import HorizontalBarGraph from '@johnsoryna/mybargraph'

function App() {
  const [value, setValue] = useState(42)
  return (
    <>
      <input type="range" min="0" max="100" value={value}
        onChange={(e) => setValue(Number(e.target.value))} />
      <HorizontalBarGraph value={value} maxvalue={100} barwidthpx={420} label="Fortschritt"
    />
      <HorizontalBarGraph value={value} maxvalue={60} barwidthpx={300} label="Ziel 60"
    color="#16a34a" />
    </>
  )
}
```

In der `package.json` des Konsumenten steht danach `"@johnsoryna/mybargraph": "^1.0.0"`. Das CSS muss nicht separat importiert werden, weil es im Bundle injiziert ist.



`node_modules/@johnsoryna/mybargraph` im Konsumenten-Projekt: nur `dist/`, `README` und `package.json`



npm run dev im Konsumenten: HorizontalBarGraph aus dem npm-Package

Aufgabe: Package vom Lernpartner einbinden

Lernpartner: GitHub-Benutzer Samusn. Package: @samusn/react-card-package, Version 1.0.0, publiziert auf GitHub Packages (github.com/Samusn/react-card-package/pkgs/npm/react-card-package).

Damit ist genau der Fall aus der Recherche oben eingetreten: Das Package liegt nicht auf npmjs.com, sondern in der GitHub-Registry. npm install würde es dort gar nicht suchen. Im Konsumenten braucht es deshalb eine .npmrc, die den Scope @samusn auf npm.pkg.github.com umleitet und ein Token mitgibt. Das Token steht nur in der Umgebungsvariable GITHUB_TOKEN, die .npmrc selbst kann ins Repository, wie die settings.xml bei Maven mit \${env.GITHUB_TOKEN}.

```
# bargraph-consumer/.npmrc
@samusn:registry=https://npm.pkg.github.com
//npm.pkg.github.com/:_authToken=${GITHUB_TOKEN}

# PowerShell: Token aus der GitHub-CLI übernehmen, dann installieren
$env:GITHUB_TOKEN = gh auth token
npm install @samusn/react-card-package@1.0.0
```

Erster Versuch: Die Registry antwortet mit 403 "permission_denied: read_package". Das Repository war privat. Nachdem der Partner das Repository auf public gestellt hatte, kam immer noch 403: Bei GitHub Packages hat das Package eine eigene Sichtbarkeit (Package settings, Danger Zone, Change visibility), die nicht automatisch mit dem Repository wechselt. Erst nach diesem zweiten Schritt liess sich das Package installieren. Auf npmjs.com wäre beides nicht nötig gewesen.

```
PS C:\Code\324\bargraph-consumer> npm install @samusn/react-card-package@1.0.0
found 0 vulnerabilities

package.json:      "@samusn/react-card-package": "^1.0.0"
package-lock.json: "resolved": "https://npm.pkg.github.com/download/@samusn/react-card-package/1.0.0/..."
node_modules/@samusn/react-card-package/dist/react-card-package.es.js
```

Das Package exportiert die Komponente Card (Named Export) mit den Props title, image, style und children. Ich verwende sie in src/PartnerDemo.jsx zweimal, in der zweiten Card steckt zusätzlich ein Balken aus meinem eigenen Package, damit beide Packages zusammen sichtbar sind:

```
// src/PartnerDemo.jsx (gekürzt)
import { Card } from '@samusn/react-card-package'
import HorizontalBarGraph from '@johnsoryna/mybargraph'

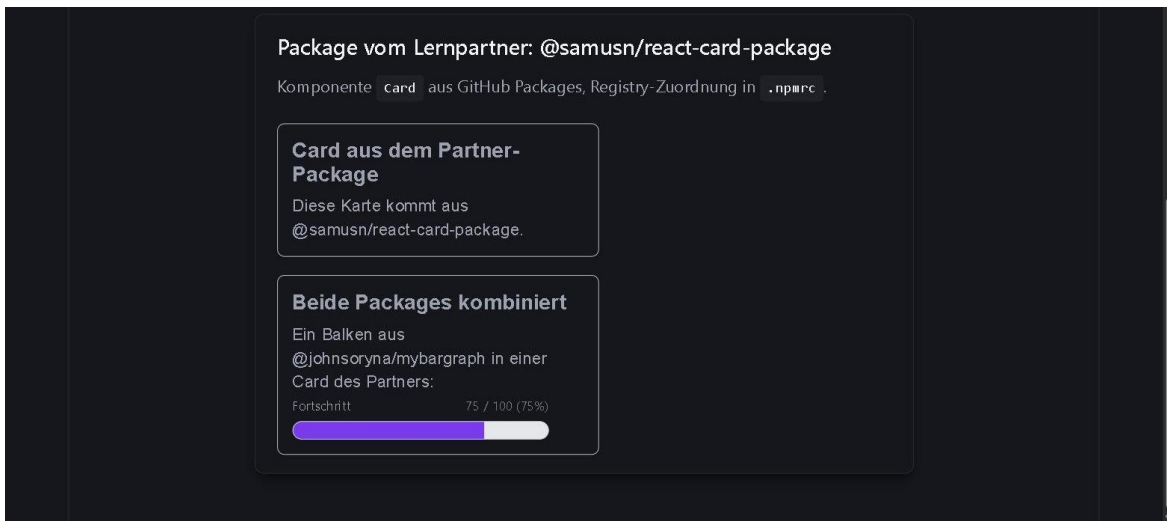
<Card title="Card aus dem Partner-Package">
  <p>Diese Karte kommt aus @samusn/react-card-package.</p>
</Card>
<Card title="Beide Packages kombiniert">
  <HorizontalBarGraph value={75} maxvalue={100} barwidthpx={280} label="Fortschritt"
  color="#7c3aed" />
</Card>
```

Problem: require ist nicht definiert

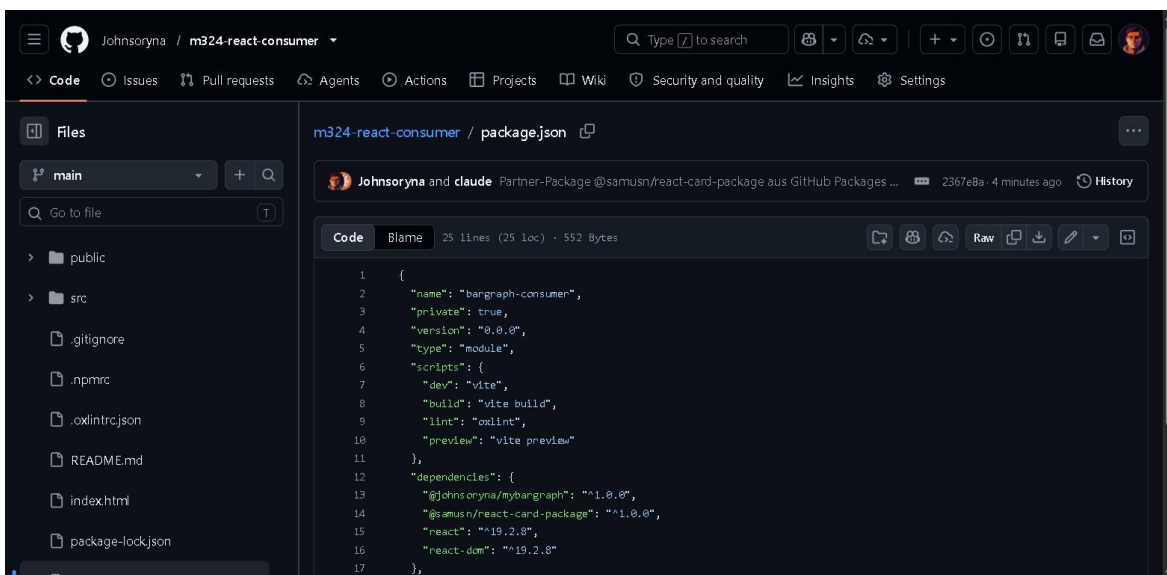
Nach dem Import blieb die Seite leer. In der Browser-Konsole: "Calling `require` for "react" in an environment that doesn't expose the `require` function". Ursache: In der vite.config.js des Partners steht unter rollupOptions.external nur react und react-dom, nicht react/jsx-runtime. Vite hat deshalb den JSX- Runtime von React mit ins Bundle kopiert, und dieser Code holt sich React per require("react"). Im Browser gibt es kein require. Mein eigenes Package hat react/jsx-runtime als external, darum tritt das Problem dort nicht auf.

Die richtige Korrektur gehört ins Package des Partners (external ergänzen, Version 1.0.1 publizieren). Damit der Konsument trotzdem läuft, habe ich ein kleines Vite-Plugin geschrieben, das nur dieser einen Datei ein lokales require gibt, das "react" liefert. Weil vorgebundelte Abhängigkeiten an Plugins vorbeigehen, muss das Package zusätzlich aus optimizeDeps ausgeschlossen werden.

```
// vite.config.js des Konsumenten (gekürzt)
function fixPartnerRequire() {
  const target = /@samusn[\\/]react-card-package[\\/]dist[\\/]react-card-package\.es\.js$/
  return {
    name: 'fix-partner-require', enforce: 'pre',
    transform(code, id) {
      if (!target.test(id.split('?')[0])) return
      return { code: "import * as __react from 'react';\n" +
        "const require = (m) => { if (m === 'react') return __react.default ?? __react;\n" +
        "throw new Error(m) };\n" + code }
    },
  }
}
export default defineConfig({
  plugins: [fixPartnerRequire(), react()],
  optimizeDeps: { exclude: ['@samusn/react-card-package'] },
})
```



Konsument: zwei Cards aus @samusn/react-card-package, in der zweiten ein HorizontalBarGraph aus @johnsoryna/mybargraph



package.json des Konsumenten auf GitHub mit beiden Packages als dependencies

Fazit und Probleme

- Ein React-Package unterscheidet sich von einer App nur durch Einstiegspunkt, Library-Build und die package.json-Felder files, exports und peerDependencies. Der Rest ist normales React.
- `__dirname` aus der Anleitung existiert in einer ESM-Konfiguration nicht, `fileURLToPath(new URL(...))` ersetzt es.
- "vite demo" findet die Konfiguration nicht, darum eine separate vite.demo.config.js.
- prop-types als Dependency, nicht als Peer, und trotzdem external im Build: npm installiert es beim Konsumenten mit, aber es wird nicht ins Bundle kopiert.
- Der Unterschied zu Maven: Bei npm ist der Konsument-Registry-Zugriff ohne Konfiguration möglich, bei GitHub Packages braucht auch npm (wie Maven) ein Token zum Lesen.

- Ein Package muss alle React-Einstiegspunkte als external markieren, auch react/jsx-runtime. Fehlt das, landet ein require("react") im Bundle und das Package funktioniert im Browser nicht. Der Konsument kann das nur mit einem Workaround umgehen, die eigentliche Korrektur gehört ins Package.
- Bei GitHub Packages sind Repository- und Package-Sichtbarkeit zwei getrennte Einstellungen.
- npmjs.com erzwingt 2FA beim Publizieren. Der Access-Token-Weg mit "Bypass 2FA" aus der Anleitung wird eingeschränkt, für Pipelines ist Trusted Publishing (OIDC) der vorgesehene Ersatz.